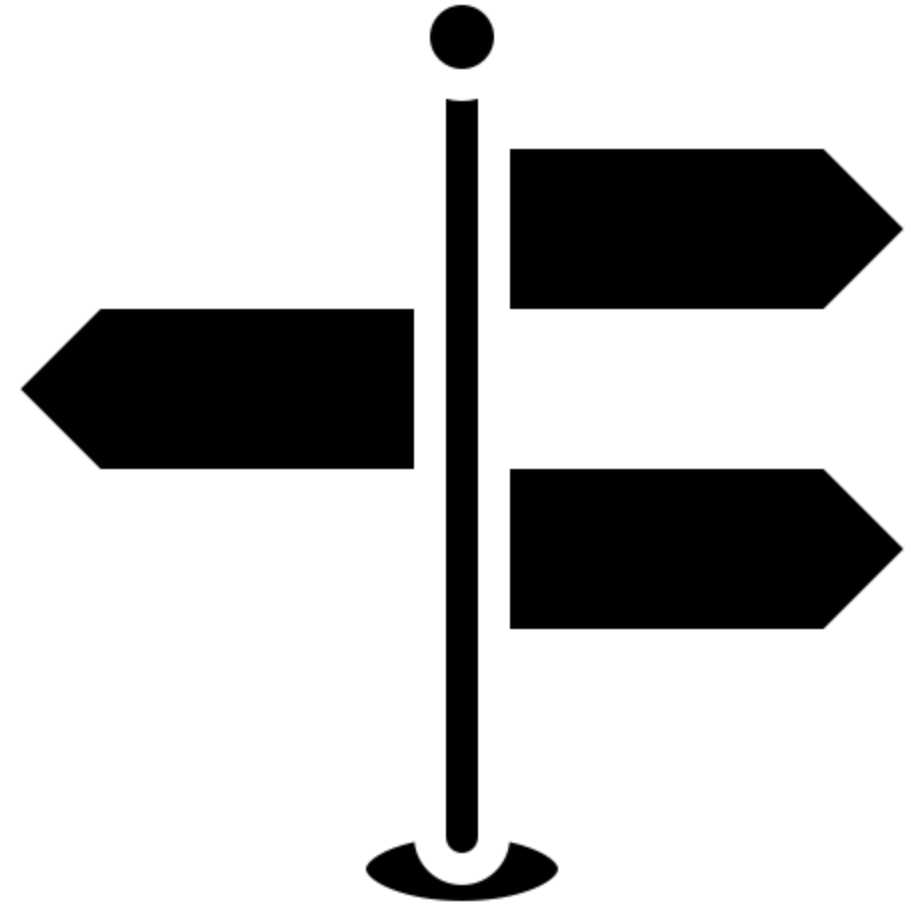


Public Cloud Services Function as a Service



Learning objectives

- Students will gain an overview of the concept of Function as a Service (FaaS) and learn about its uses.
- Students will be able to distinguish FaaS from other service levels such as PaaS and IaaS and use it according to requirements.
- Students will understand the advantages and disadvantages of FaaS.



Motivation

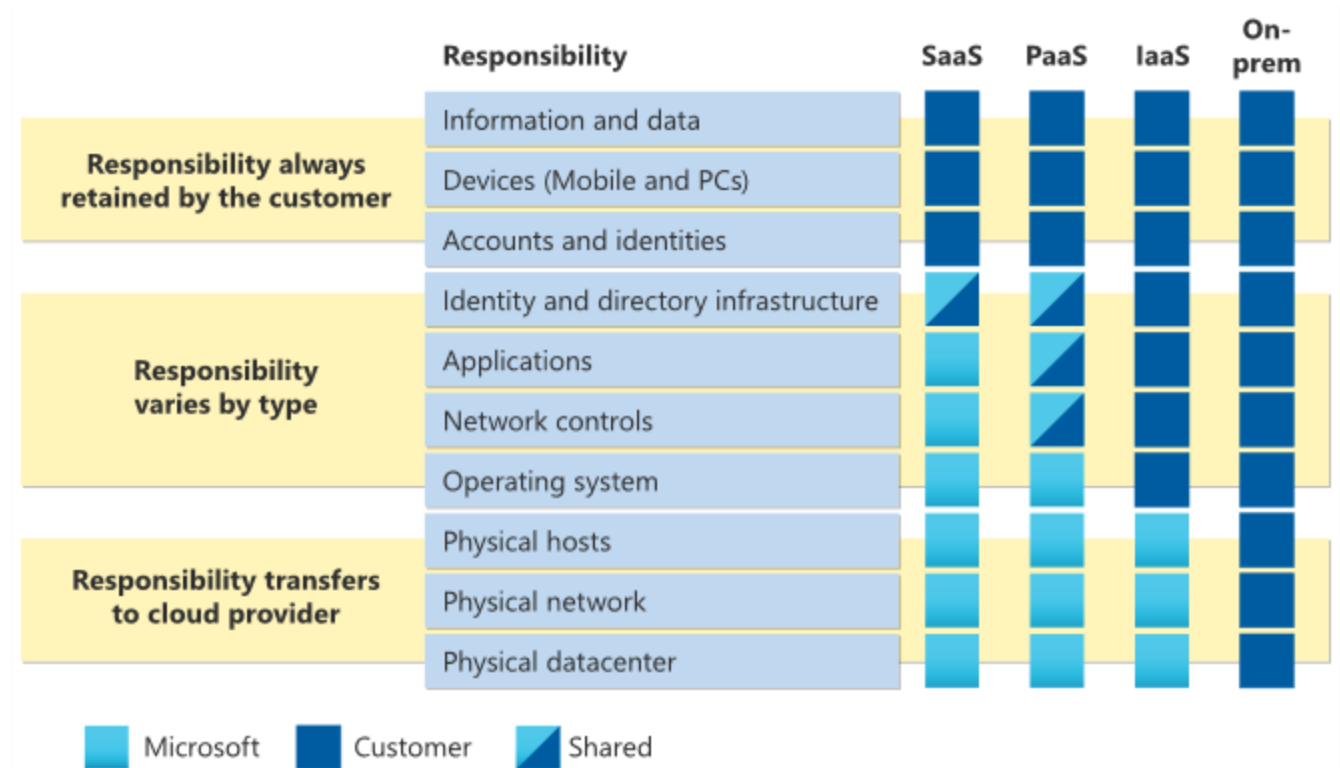


Agenda

- Fundamentals and concept of Function as a Service (FaaS)
- Challenges of FaaS
- Patterns and use cases for FaaS
- FaaS service offerings, tools, and solutions
- SDKs and integrations in code
- Deployment and operation
- Pricing and vendor lock-in

Function as a Service - Fundamentals

- VMs -> Container Services -> Function as a Service (FaaS)
- No more underlying platform maintenance
- Focus on small, reusable code
 - upload it and the runtime takes care of the rest
- Every cloud platform offers many native triggers for starting functions



<https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>

Function as a Service – Serverless

«Serverless computing refers to the concept of building and running applications that do not require server management.»(1)

- Functions are a serverless technology
- Serverless is applied to other areas such as cloud databases with serverless SKUs

Features of serverless platforms (1)

No operation of servers

No installation, upgrade, or maintenance required

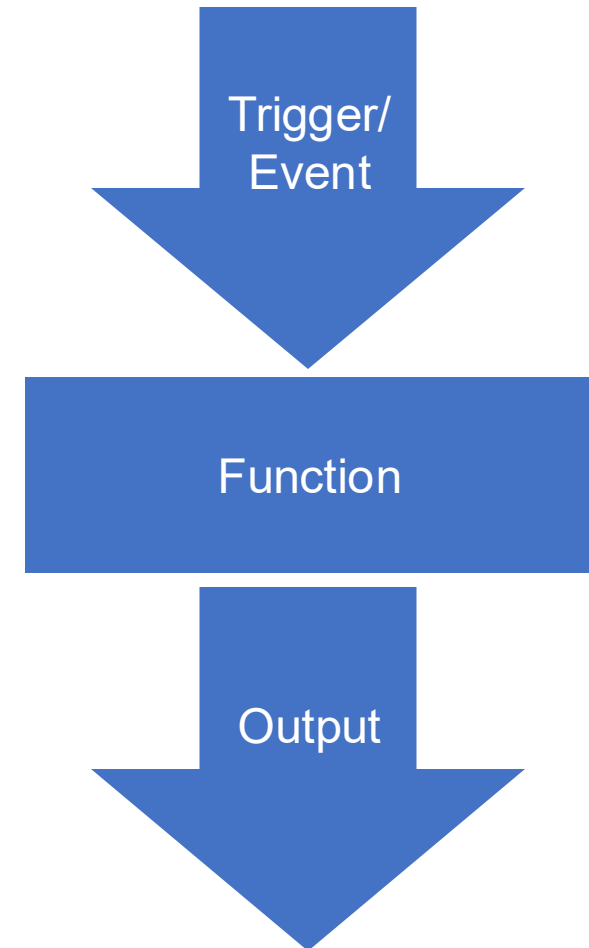
Flexible scaling of resources

No costs without traffic

https://github.com/cncf/wg-serverless/blob/master/whitepapers/serverless-overview/cncf_serverless_whitepaper_v1.0.pdf (1)

Function as a Service – Fundamentals

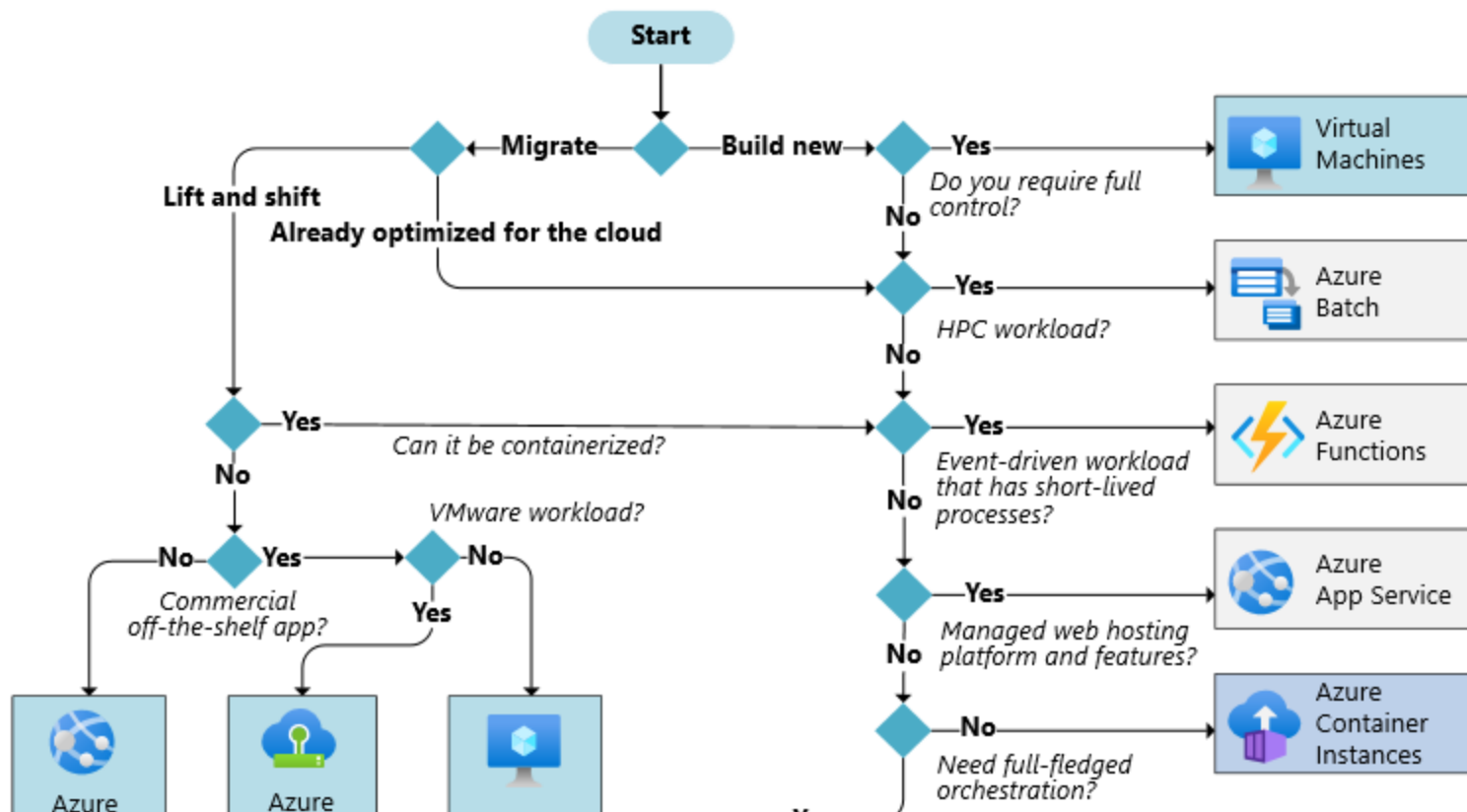
- Features
 - Short-lived
 - Small resource footprint
 - Started by trigger or event
 - Optional output
 - Authentication/authorization levels
- Differentiation from IaaS and PaaS (decision tree)
<https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/compute-decision-tree>)



Function as a Service – Fundamentals

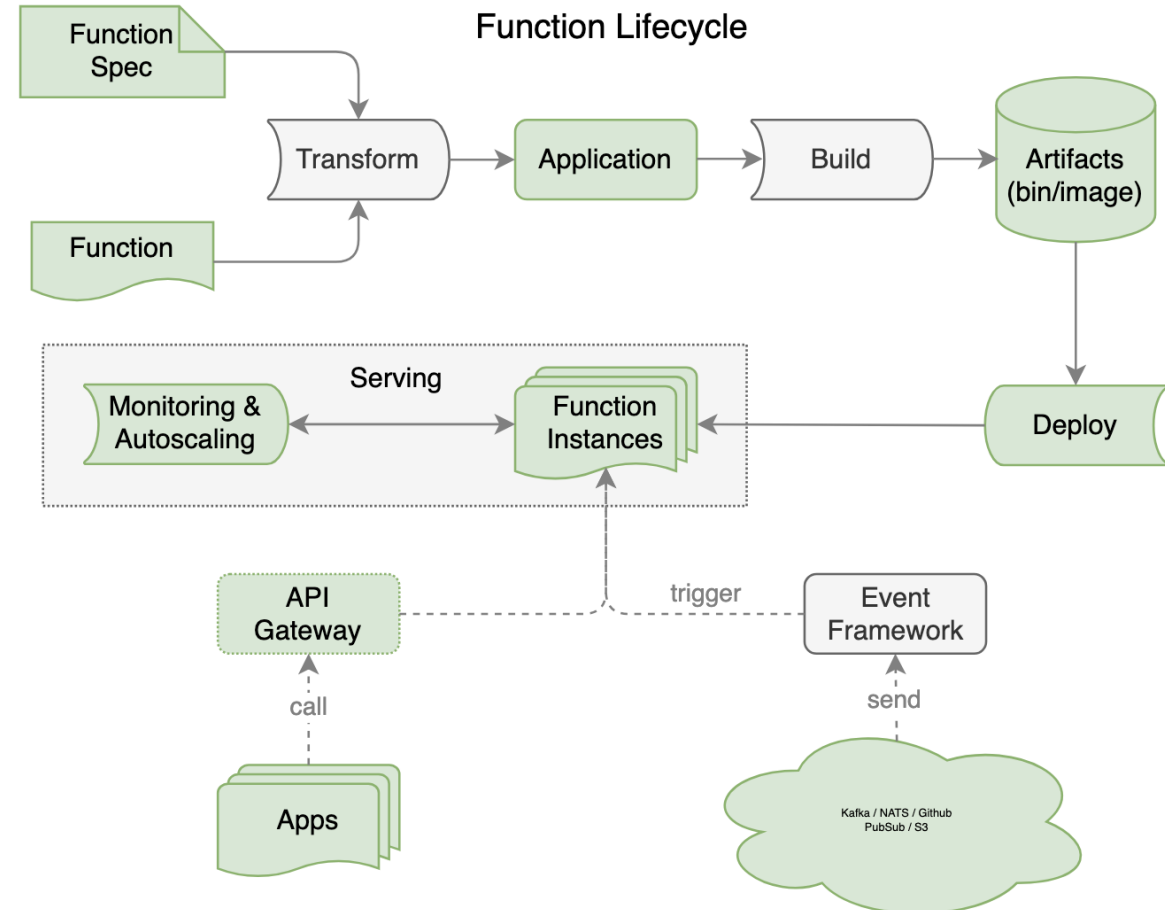
Differentiation from IaaS and PaaS

(decision tree <https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/compute-decision-tree>)



Function as a Service – Serverless Pipeline

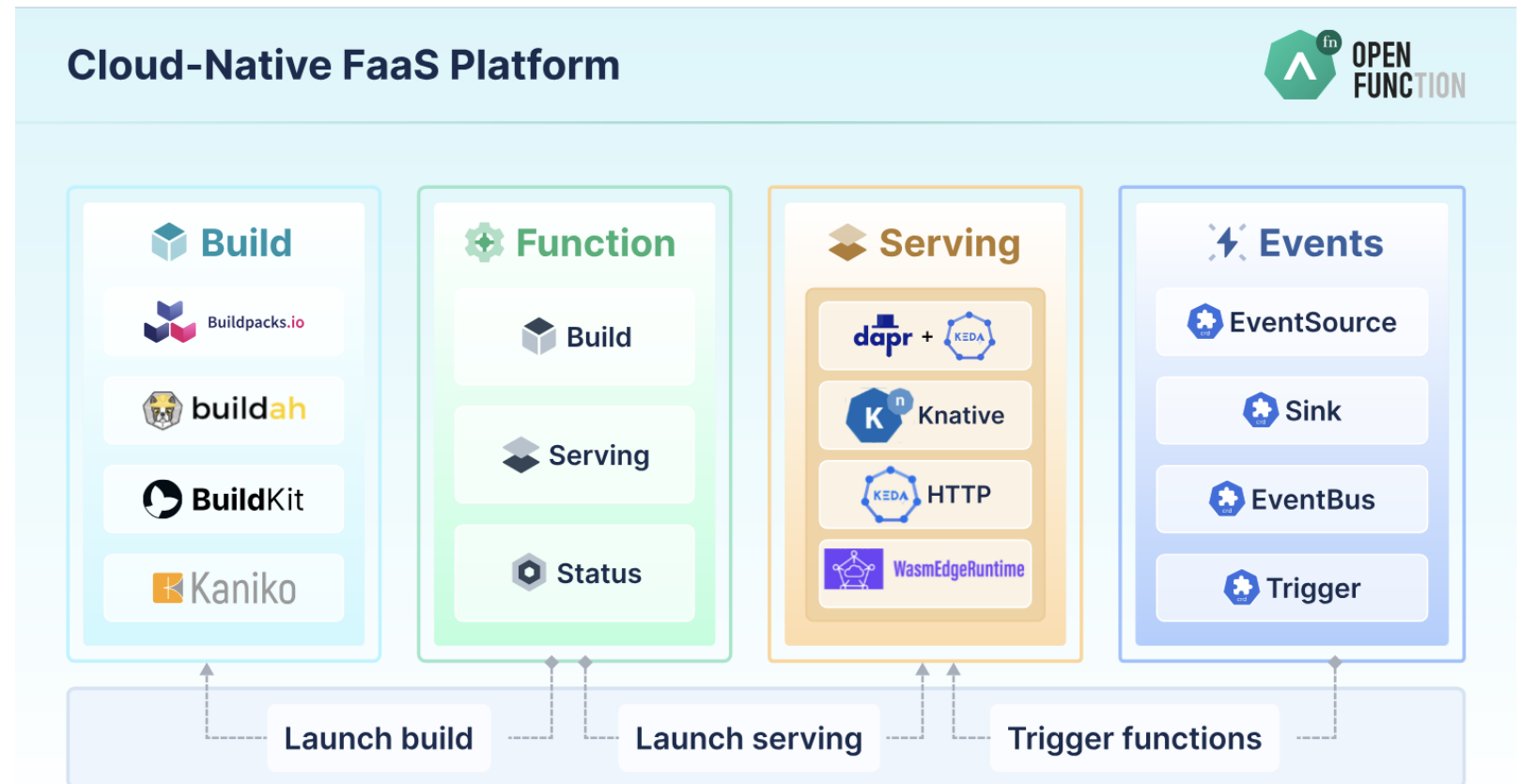
- Code is packaged and loaded onto the runtime
- Runtime listens for triggers and executes the code depending on the configuration
- The runtime takes care of all lifecycle and operational aspects (e.g., monitoring)



<https://openfunction.dev/docs/introduction/>

Function as a Service – Serverless Pipeline with Tooling

- Platforms leverage additional services for a full FaaS service
- I.E. Build via Buildpacks or Cloud Build



<https://openfunction.dev/docs/introduction/>

Function as a Service – Runtimes

- Scope of support varies (triggers, bindings, etc.) per language
- Deployment as
 - Code (zip upload)
 - Docker container

Choose a programming language

C# Java JavaScript PowerShell **Python** TypeScript

This article explains the levels of support offered for your preferred language when using Azure Functions. It also describes strategies for creating functions using languages not natively supported.

There are two levels of support:

- **Generally available (GA)** - Fully supported and approved for production use.
- **Preview** - Not yet supported, but expected to reach GA status in the future.

Languages by runtime version

Make sure to select your preferred development language at the [top of the article](#).

The following table shows the language versions supported for Python functions.

 Expand table

Supported version	Support level	Expected end-of-support date
Python 3.13	Preview	October 2029
Python 3.12	GA	October 2028
Python 3.11	GA	October 2027
Python 3.10	GA	October 2026
Python 3.9	GA	October 2025

For more information, see [Azure Functions Python developer guide](#).

For information about planned changes to language support, see the [Azure roadmap updates](#).

<https://learn.microsoft.com/en-us/azure/azure-functions/supported-languages?tabs=isolated-process%2Cv4&pivots=programming-language-python>

Azure Function Deployment Plans

Example: Azure Functions

- Azure Functions require a runtime (function host/app service).
- Runtimes can be executed on various platforms such as app service plans, container apps, or Kubernetes.
- Runtimes may restrict operating systems and/or features.

Hosting option	Service	Availability	Container support
Flex Consumption plan	Azure Functions	Generally available (GA)	None
Premium plan	Azure Functions	GA	Linux
Dedicated plan	Azure Functions	GA	Linux
Container Apps	Azure Container Apps	GA	Linux
Consumption plan	Azure Functions	GA	None

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale>

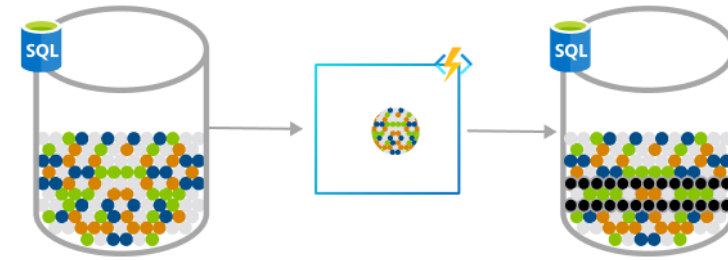
Function as a Service - Challenges

- **State handling:** Functions have no state (no session, etc.). This must be stored separately externally (e.g., in a database).
- **Cold Starts:** Various programming languages have longer startup times. The first trigger that “wakes up” the function has to wait longer. The code can already be loaded, which usually entails additional costs.
- **Orchestration:** Due to the lightweight nature of functions, it is common for several functions to work on a higher-level task.

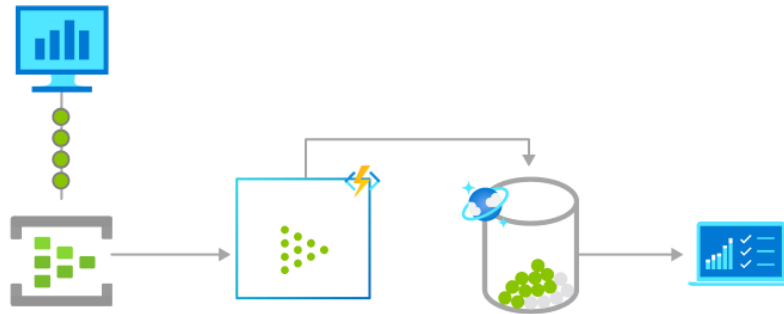
Function as a Service – Use Cases



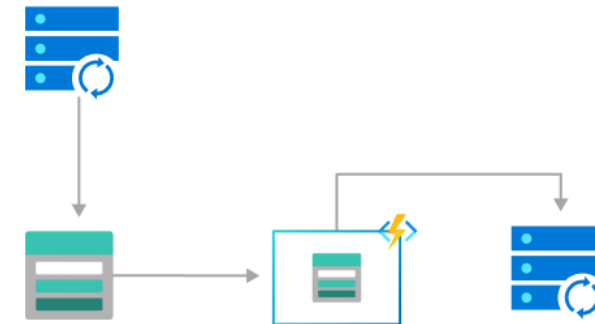
Web APIs



Scheduled Tasks



Streaming

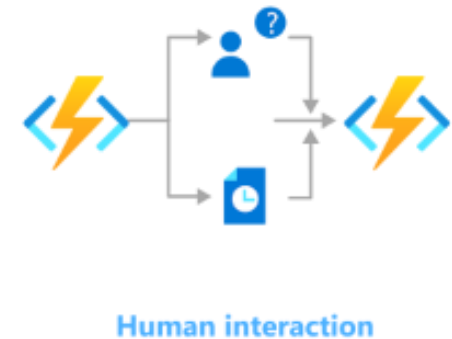
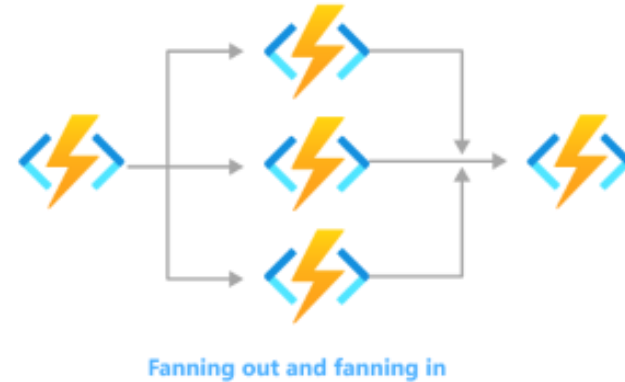


File Processing

More : <https://learn.microsoft.com/en-us/azure/azure-functions/functions-scenarios?pivots=programming-language-csharp>

Function as a Service - Scenarios

- Combination of functions and services
- Parallelization or serialization
- Decoupling of process steps



<https://learn.microsoft.com/en-us/azure/azure-functions/functions-scenarios?pivots=programming-language-csharp>

Service-Offerings, Tools und Solutions – Categories

- Low-Code Power Automate, Logic Apps etc. (<https://learn.microsoft.com/en-us/azure/azure-functions/functions-compare-logic-apps-ms-flow-webjobs>), AWS Step Functions
- Cloud- and Code-Based services – Azure Function, AWS Lambda etc.
- Kubernetes and Container based e.g. Knative or KEDA
- Platformagnostic e.g. Openwhisk

Service-Offerings, Tools und Solutions – Landscape

Serverless Landscape by CNCF <https://landscape.cncf.io/?group=serverless>

Serverless

Installable Platform

CNCF GRADUATED

CNCF GRADUATED

CNCF INCUBATING

Embedded Functions

CNCF GRADUATED

Hosted Platform

Framework

CNCF GRADUATED

Tools

Installable Platforms

Service-Offerings, Tools und Solutions – Technical Baseline

- Function as a Service solutions have different technical foundations – from custom solutions (Firecracker/AppService) to well-known open source projects (Kubernetes).
- Compatibility between platforms may be possible, e.g., Google Cloud Run and Knative.
- Compatibility between custom solutions is almost non-existent (requires refactoring during migration if no framework such as Flask was used).



AWS Lambda

AWS Firecracker



Azure Functions

Azure AppService



Others (e.g. Mesos)



Knative

Kubernetes

AWS Lambda in Comparison to Azure Functions

Thema	AWS Lambda	Azure Functions
Runtimes	Java, Go, PowerShell, Node.js, C#, Python, Ruby and Bring your Own	C#, Java, JavaScript, Python and PowerShell
Exposure to users	Direct or via API Gateway	Direct or via Azure Application Gateway / API-Management
Stateful functions	AWS Step Functions	Azure Durable Functions



Similar concepts on both platforms regarding trigger/events, bindings, networking or IAM. High degree of lock-in.

SDKs and Code-Integrations with Python on Azure

- Decorators are used to define triggers and bindings (HTTP in this case).
- The entry point is the file `function_app.py`.

```
import azure.functions as func

app = func.FunctionApp()

@app.function_name(name="HttpTrigger1")
@app.route(route="req")
def main(req: func.HttpRequest) -> str:
    user = req.params.get("user")
    return f"Hello, {user}!"
```

Source: <https://learn.microsoft.com/en-us/azure/azure-functions/functions-reference-python?tabs=get-started%2Casgi%2Capplication-level&pivots=python-mode-decorators#programming-model>

SDKs and Code-Integrations with Python on Azure

- Recommended folder structure (may vary)
- Important: Azure must be able to load dependencies and know the application's entry point.

```
<project_root>/
| - .venv/
| - .vscode/
| - function_app.py
| - additional_functions.py
| - tests/
| | - test_my_function.py
| - .funcignore
| - host.json
| - local.settings.json
| - requirements.txt
| - Dockerfile
```

Source: <https://learn.microsoft.com/en-us/azure/azure-functions/functions-reference-python?tabs=get-started%2Casgi%2Capplication-level&pivots=python-mode-decorators#programming-model>

SDKs and Code-Integrations with Python on AWS

- The entry point is the file `lambda_function.py`
- Main entry point `“lambda_handler”`
- The “event” object contains predefined fields depending on the trigger

```
import sys
import json

def lambda_handler(event, context):
    print('Received event: ' + json.dumps(event, indent=2))
    print('Context: ' + str(context))

    """
    Main Lambda handler function
    Parameters:
        event: Dict containing the Lambda function event data
        context: Lambda runtime context
    Returns:
        Dict containing status message
    """
    try:
        # Parse the input event
        order_id = event['Order_id']
        amount = event['Amount']
        item = event['Item']
```

Source: <https://docs.aws.amazon.com/lambda/latest/dg/lambda-python.html>

SDKs and Code-Integrations with Python on Knative

- All functions are packaged as containers
- “service.yaml” Describes the context (and any mappings) of the function
- Example based on Flask – no dedicated SDK

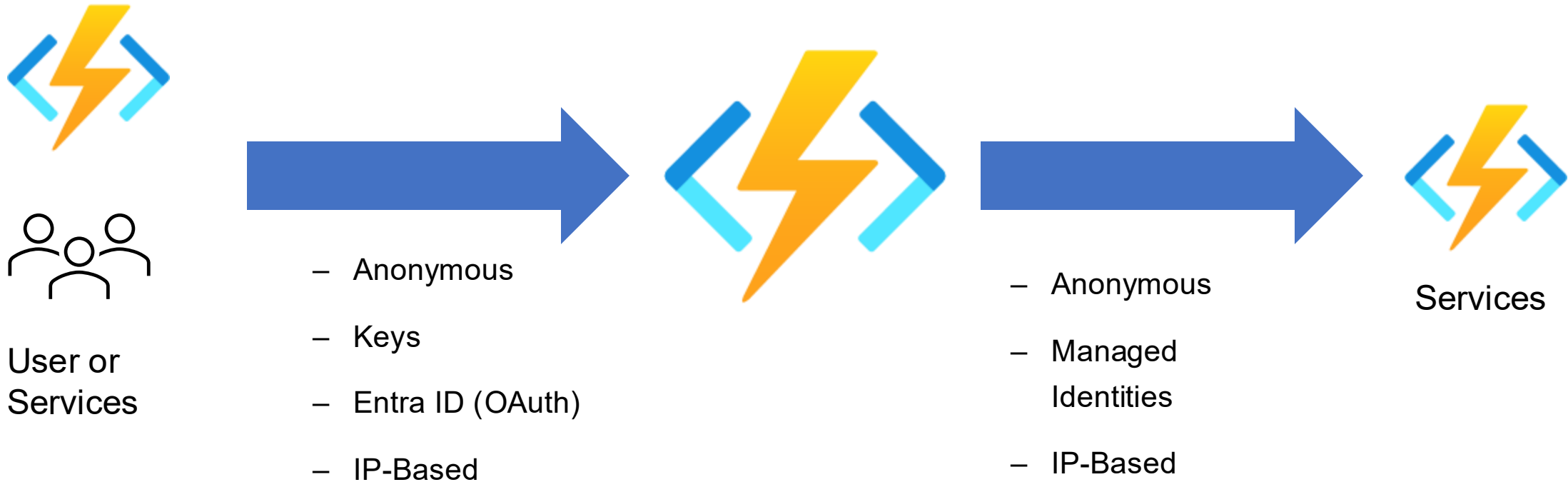
```

apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: helloworld-python
  namespace: default
spec:
  template:
    spec:
      containers:
        - image: docker.io/{username}/helloworld-python
          env:
            - name: TARGET
              value: "Python Sample v1"

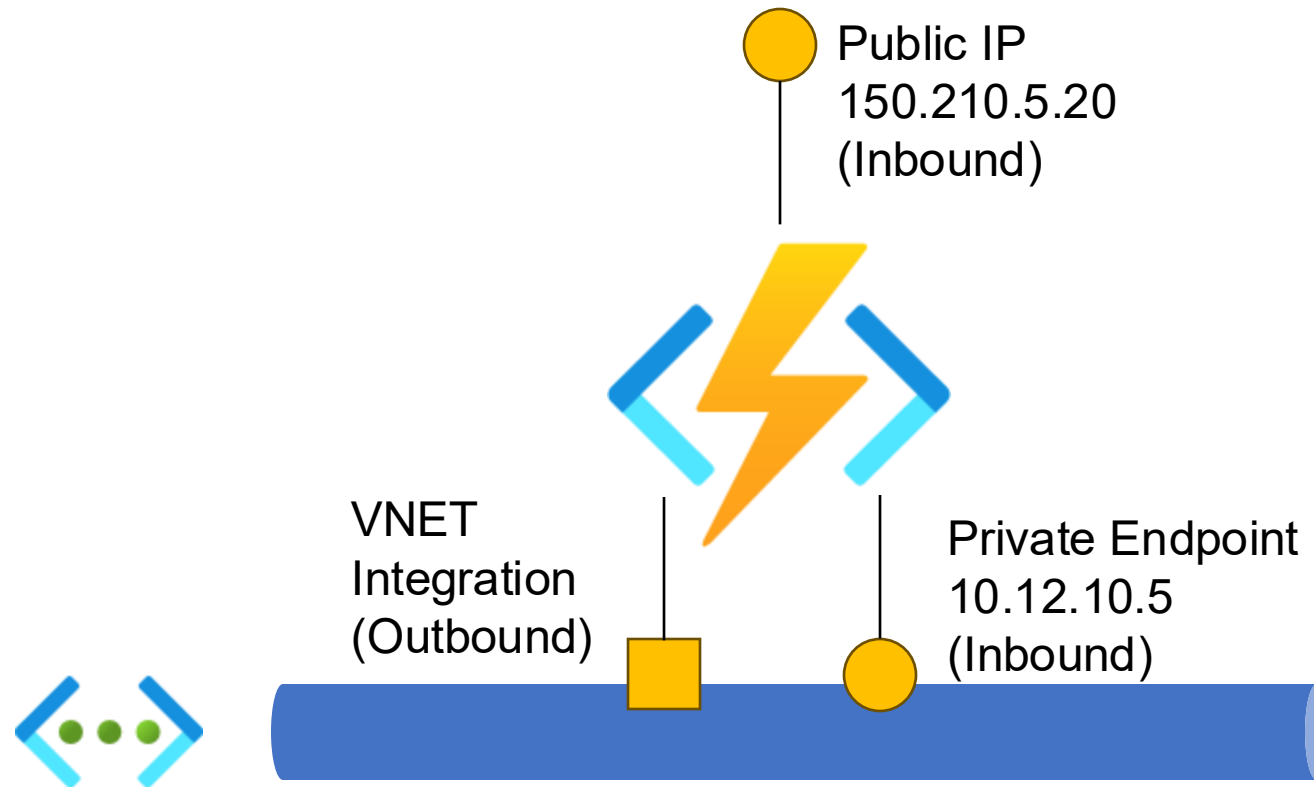
```

Source: <https://github.com/knative/docs/tree/main/code-samples/serving/hello-world/helloworld-python>

Azure Functions Authentication and Authorization



Azure Functions Networking



Function as a Service – Walkthrough

- Log in to <https://portal.azure.com>
- Navigate to your own resource group
- Create a new resource – search for Function
- Create a new Function App
- Select Flex Plan

[Home](#) > [p-sub-htimvs-pcls](#) | [Resource groups](#) > [mme-function-demo](#) >

Marketplace

Get Started

Service Providers

Management

Private Marketplace

Private Offer Management

My Marketplace

Favorites

My solutions

Recently created

Private plans

Categories

IT & Management Tools (88)

Compute (61)

Security (55)

☐ Azure benefit eligible only ⓘ ☐ Azure services only

 **New!** Get AI-generated suggestions for 'function'

Showing 1 to 20 of 303 results for 'function'. [Clear search](#)



Function App

Microsoft

Azure Service

Create an Azure Functions app, where you run event-driven code in a serverless environment. Build a web API, respond to database changes, process live event streams, and

Create ▾ 



Azure Network Function Manager - Device

Microsoft

Azure Service

Device for Azure Network Function Manager

Create ▾ 

Function as a Service – Walkthrough

- Assign name
- Use Switzerland North as region
- Python for runtime
- Create (rest can remain default)

[Home](#) > [p-sub-htimvs-pcls](#) | [Resource groups](#) > [mme-function-demo](#) > [Marketplace](#) > [Create Function App](#) >

Create Function App (Flex Consumption) ...

[Basics](#) [Storage](#) [Azure OpenAI](#) [Networking](#) [Monitoring](#) [Deployment](#) [Authentication](#) [Tags](#) [Review + create](#)

Create a function app, which lets you group functions as a logical unit for easier management, deployment and sharing of resources. Functions lets you execute your code in a serverless environment without having to first create a VM or publish a web application.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ

Resource Group * ⓘ [Create new](#)

Instance Details

Function App name * [.azurewebsites.net](#)

☒ Try a secure unique default hostname (preview). [More about this update](#) ↗

Region *

Runtime stack *

Version *

Instance size * ⓘ

SDKs and Code-Integrations – Links

- Native Python Azure Functions SDK:
<https://learn.microsoft.com/de-de/azure/azure-functions/functions-reference-python?tabs=get-started%2Casgi%2Capplication-level&pivots=python-mode-decorators>
- Fastapi Example: <https://learn.microsoft.com/en-us/samples/azure-samples/fastapi-on-azure-functions/fastapi-on-azure-functions/>
- Quarkus Example:
<https://quarkus.io/guides/azure-functions>

```
import azure.functions as func

from WrapperFunction import app as fastapi_app

app = func.AsgiFunctionApp(app=fastapi_app, http_auth_level=func.AuthLevel.ANONYMOUS)
```

```
import azure.functions as func

import fastapi

app = fastapi.FastAPI()

@app.get("/sample")
async def index():
    return {
        "info": "Try /hello/Shivani for parameterized route.",
    }
```

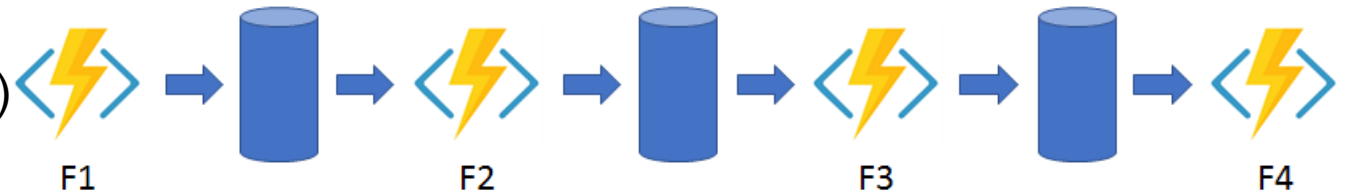
Error Handling

- Retries (e.g., from blob triggers)
 - # Attempts
 - Backoff algorithm
- Logging
 - Open Telemetry
 - Stdout
 - Direct integration

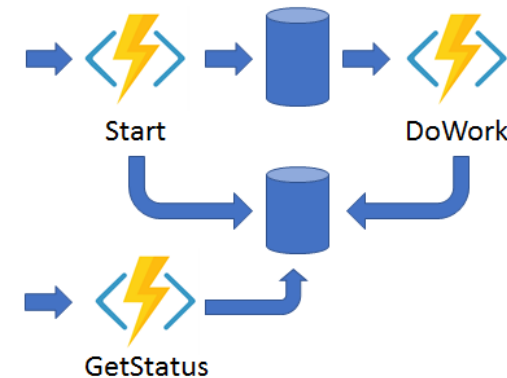
```
@app.timer_trigger(schedule="*/1 * * * * *",  
                    arg_name="func_timer",  
                    run_on_startup=False,  
                    use_monitor=False)  
@app.retry(strategy="fixed_delay",  
           max_retry_count="5",  
           delay_interval="00:00:05")  
def timer_function(func_timer: TimerRequest, context: Context) → None:  
    logging.info(f'Retry number: {context.retry_context.retry_count}')
```

Durable Functions on Azure

- Extension of Azure Functions to implement stateful functions (code-first)
- Enables orchestration of function calls
- Functions can track the status of calls from other functions
- Advantages
 - Sequence
 - Retries
 - Asynchronous or parallel workflows



Function
Chaining



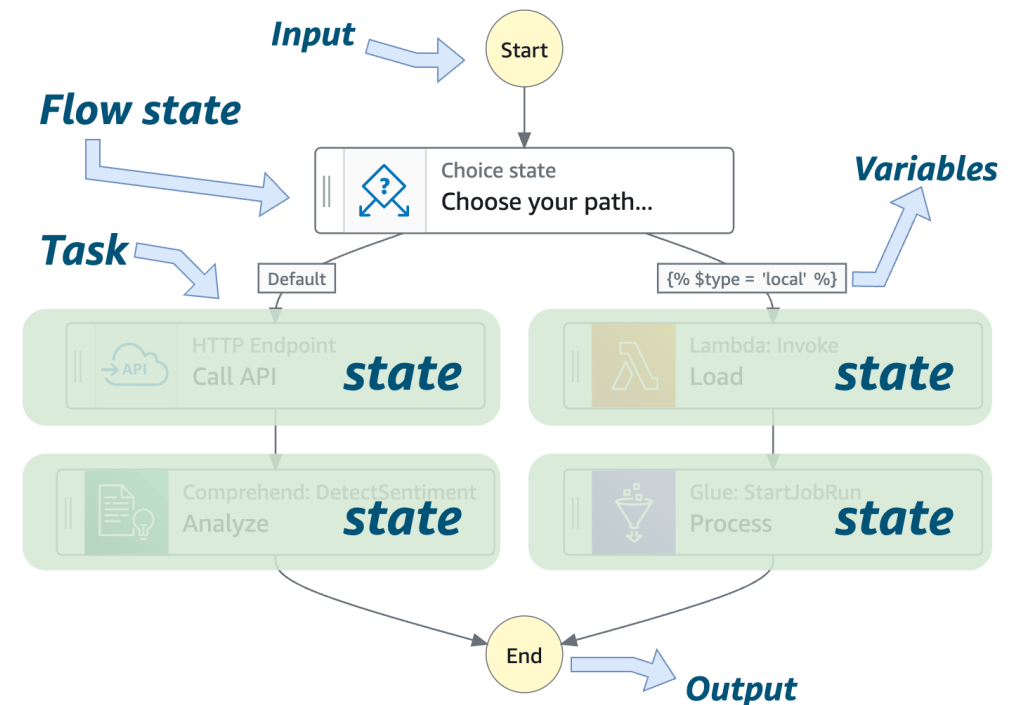
Async API

<https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview?tabs=in-process%2Cnodejs-v3%2Cv1-model&pivots=python>

Step Functions in AWS

- AWS service for orchestration of workflows
- Declarative and UI-based approach to defining workflows (effectively JSON file)
- Workflows are mapped via Statemachine
- Supports additional services besides Lambdas

State Machine / Workflow (JSONata)



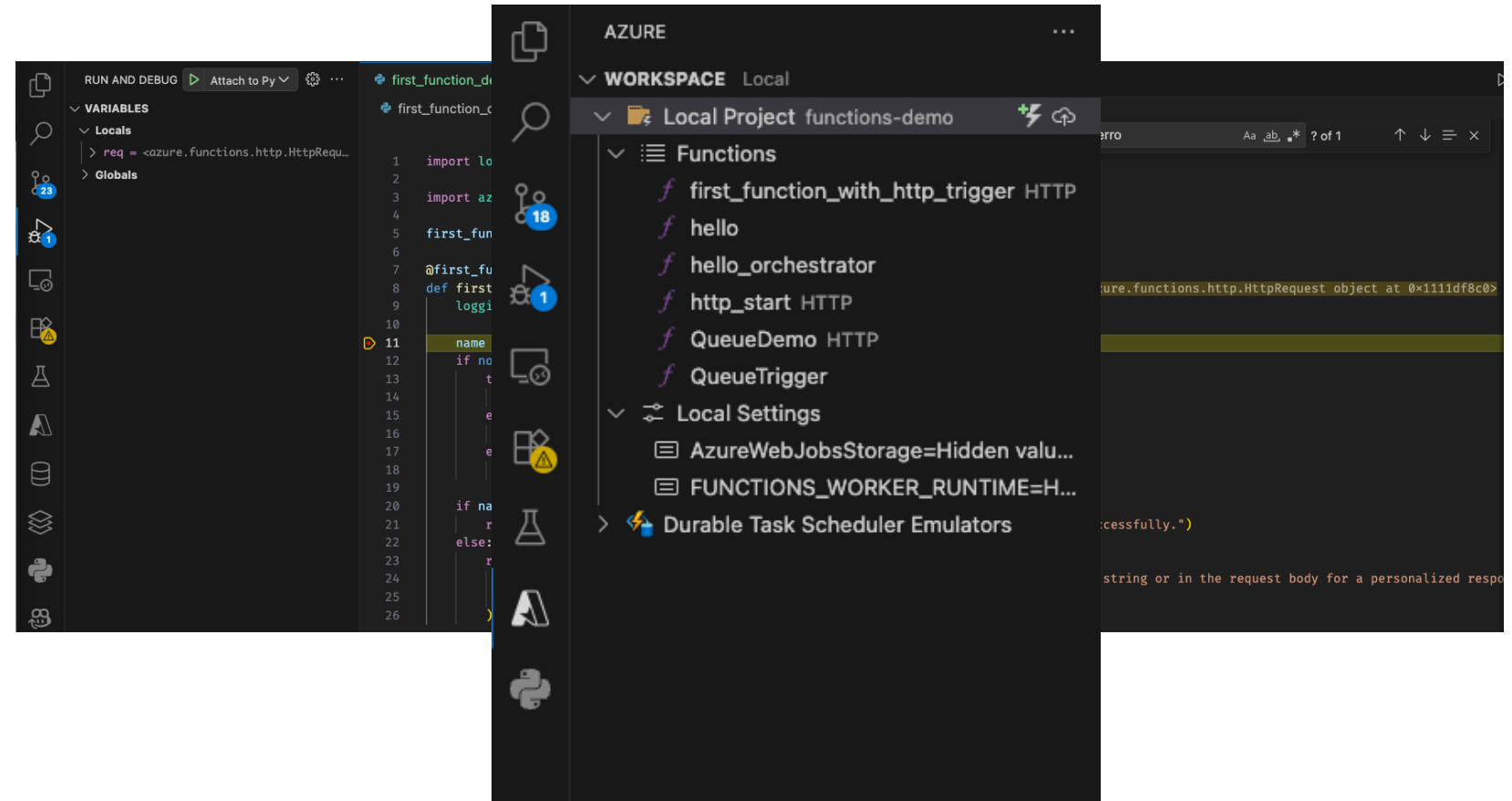
<https://docs.aws.amazon.com/step-functions/latest/dg/getting-started.html>

Function as a Service – Durable Functions

Example & Demo

Local Development with Azure

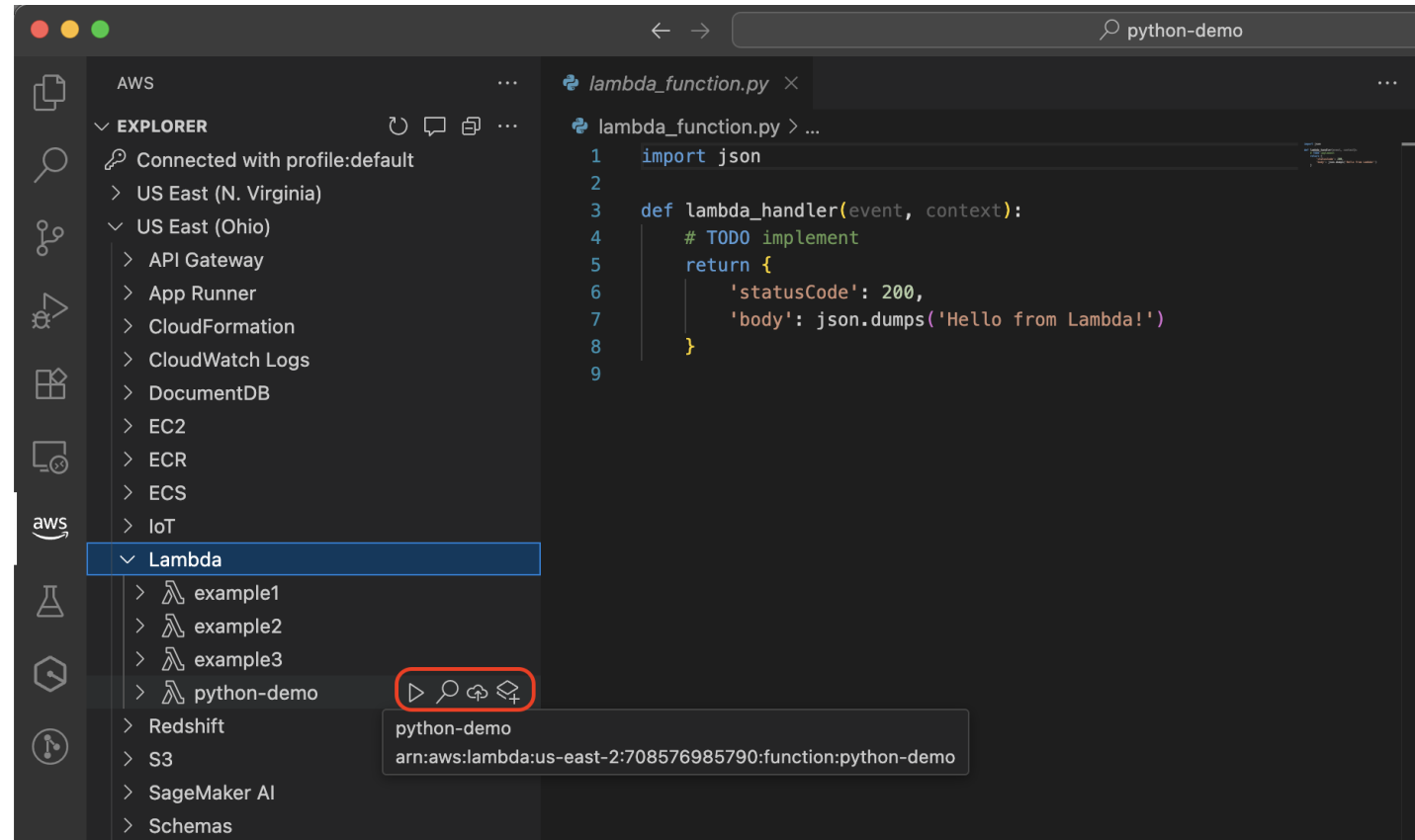
- Required tools:
 - Editor (e.g., vscode)
 - Function Runtime
 - Plugin (e.g., vscode functions extension)
 - Azure CLI
- Debugging the function code
- Local development with local (DB) emulators



<https://learn.microsoft.com/en-us/azure/azure-functions/functions-run-local?tabs=macos%2Cisolated-process%2Cnode-v4%2Cpython-v2%2Chttp-trigger%2Ccontainer-apps&pivots=programming-language-csharp>

Local Development with AWS

- Required tools:
 - Editor (e.g. vscode)
 - AWS Toolkit for vscode
 - AWS SAM CLI, AWS CDK and AWS credentials
- Direct integration in AWS SAM (Serverless Application Model)
- Local development similar to local development with Azure



Azure Functions Deployment Options

- From IDE
- Via automation, e.g.,
CI/CD pipeline
- Via CLI
- Service-managed (e.g.,
with Git push)

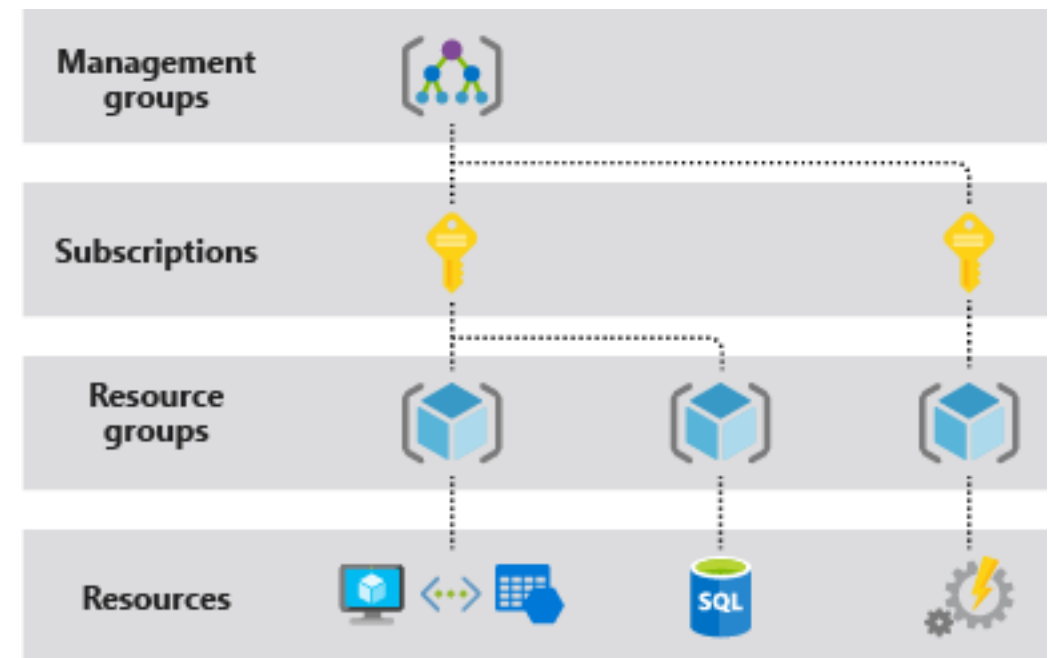
Deployment type	Methods	Best for...
Tools-based	• Azure CLI • Visual Studio Code publish • Visual Studio publish • Core Tools publish	Deployments during development and other improvised deployments. Deploying your code on-demand using local development tools .
App Service-managed	• Deployment Center (CI/CD) • Container deployments	Continuous deployment (CI/CD) from source control or from a container registry. Deployments are managed by the App Service platform (Kudu).
External pipelines	• Azure Pipelines • GitHub Actions	Production pipelines that include validation, testing, and other actions that must be run as part of an automated deployment. Deployments are managed by the pipeline.

Specific deployments should use the best technology based on the specific scenario. Many of the deployment methods are based on [zip deployment](#), which is recommended for deployment.

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-deployment-technologies?tabs=windows>

Function Deployment in Azure

- ARM: Azure Resource Manager
- Bicep: Abstraction layer on ARM
- ARM is Azure's deployment engine.
- All tools work via ARM, whether Terraform, Bicep, ARM, CLI, or others.
- ARM templates can be deployed at various levels, such as subscriptions or resource groups.
- ARM and Bicep are declarative.



<https://learn.microsoft.com/de-de/azure/azure-resource-manager/management/overview>

Deployment via Bicep in Azure

- Bicep compared to ARM is
 - Significantly more concise
 - No JSON
 - More readable
- Bicep is compiled by Azure into ARM templates during deployment

```
param location string = resourceGroup().location
param storageAccountName string = 'toylaunch${uniqueString(resourceGroup().id)}'

resource storageAccount 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
  properties: {
    accessTier: 'Hot'
  }
}
```

<https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview?tabs=bicep>

Deployment via Terraform in Azure

Important: IaC solutions focus on the infrastructure, but not on the app deployment itself. It is possible to do it (i.e. deploy from storage account or container registry)

- App deployment procedure (i.e. via Zip, Git or Container) may be configured via IaC (see https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/linux_function_app)

```
# Create a service plan
resource "azurerm_service_plan" "example" {
  name                = coalesce(var.asp_name, random_string.name.result)
  resource_group_name = azurerm_resource_group.example.name
  location            = azurerm_resource_group.example.location
  sku_name            = "FC1"
  os_type             = "Linux"
}

# Create a function app
resource "azurerm_function_app_flex_consumption" "example" {
  name                = coalesce(var.fa_name, random_string.name.result)
  resource_group_name = azurerm_resource_group.example.name
  location            = azurerm_resource_group.example.location
  service_plan_id     = azurerm_service_plan.example.id

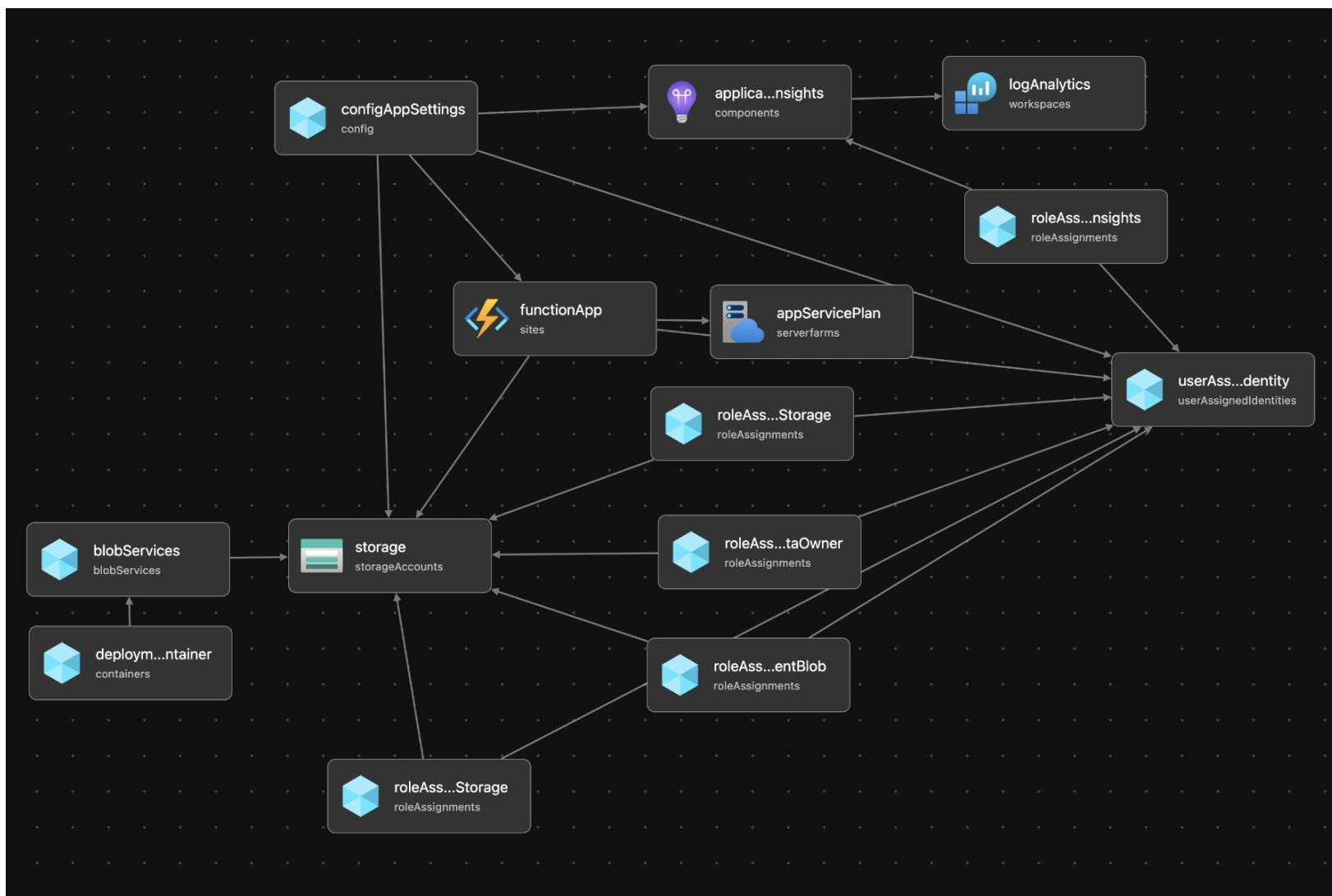
  storage_container_type      = "blobContainer"
  storage_container_endpoint  = "${azurerm_storage_account.example.primary_blob_endpoint}"
  storage_authentication_type = "StorageAccountConnectionString"
  storage_access_key          = azurerm_storage_account.example.primary_access_key
  runtime_name                = var.runtime_name
  runtime_version              = var.runtime_version
  maximum_instance_count      = 50
  instance_memory_in_mb       = 2048

  site_config {
  }
}
```

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-create-first-function-terraform?tabs=core-tools&pivots=programming-language-python>

Resource Dependencies for Functions

- Visualization: CMD + P
“Bicep: Show Visualizer”
- Resource map with dependencies and connections between resources



Deployment via IDE in Azure

- Deployment from IDE: CMD + P “Bicep Show Dataplane”
- Scope: Select subscription and resource group
- Fill in parameters
- Start validation & deployment
- What-if: Recognizes existing resources and can specify Create/Update/Delete

^ **DEPLOYMENT SCOPE**

Subscription Id
Resource Group

Change Scope

^ **PARAMETERS**

Pick JSON Parameters File

location

[resourceGroup().location]

functionAppRuntime

python

functionAppRuntimeVersion

3.11

maximumInstanceCount

40

instanceMemoryMB

2048

resourceToken

[toLower(uniqueString(subscription().id, parameter

appName

[format('func-{0}', parameters('resourceToken'))]

^ **ACTIONS**

Deploy

Validate

What-If

Deployment via CLI in Azure

- Bicep Deployment via CLI
- Use in the context of CI systems

```
# Login
az login -t fhnw365.onmicrosoft.com

# Create Resource Group
az group create --name pcls-func-iac-demo --location "Switzerland North"

az deployment group create --name pcls-func-deploy-demo --resource-group pcls-func-iac-demo --template-file
./infra/function-resources.bicep --parameters functionAppRuntime=python functionAppRuntimeVersion=3.10
```

- Function deployment in a separate pipeline

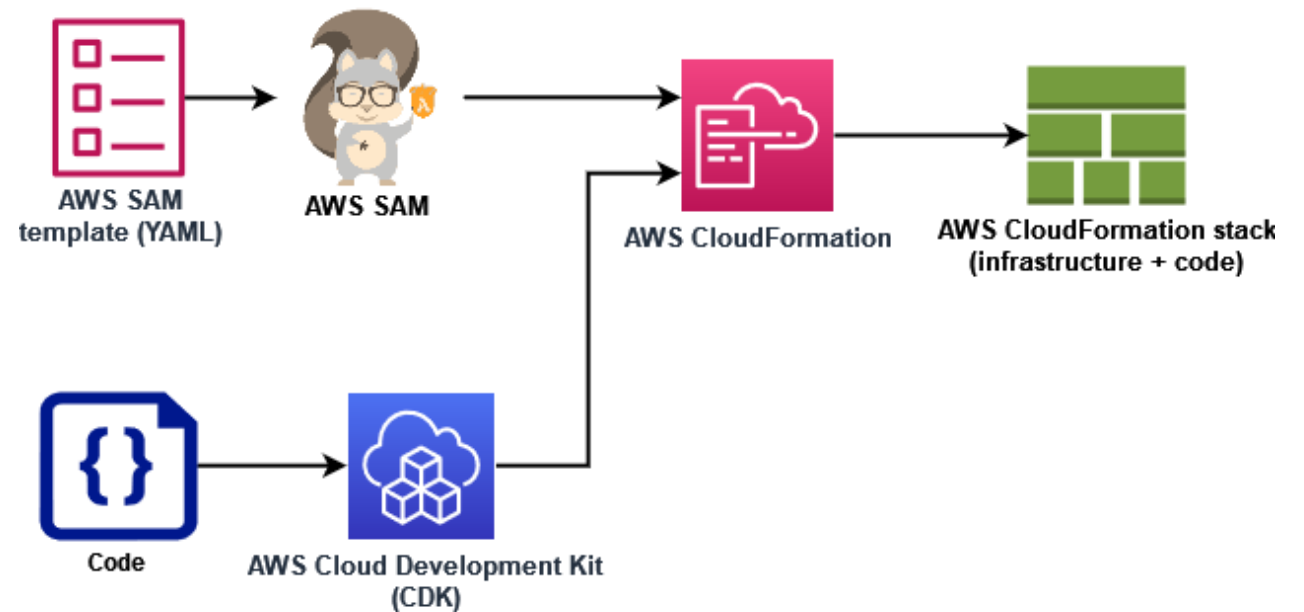
```
func azure functionapp publish <APP_NAME>
```

Deployment via Infrastructure as Code in Azure

- Declarative approach instead of imperative commands
- Greater transparency (all resources that are created are visible)
- Full control over the resources created and their configuration
- CLI: Good for rapid iteration and testing
- ARM/Bicep/Terraform: Good for maintainability, versioning, and reproducibility

Deployment via Code/CLI in AWS

- Two alternative approaches for deployment
 - AWS SAM
 - AWS CDK
- Both approaches result in CloudFormation templates and a CloudFormation stack.



<https://docs.aws.amazon.com/lambda/latest/dg/foundation-iac.html>

Azure Functions in the Azure Portal

Per function

- Inputs/triggers and outputs
- Authentication key
- Quasi-IDE in the cloud portal (e.g. test your code)
- Function logs and metrics

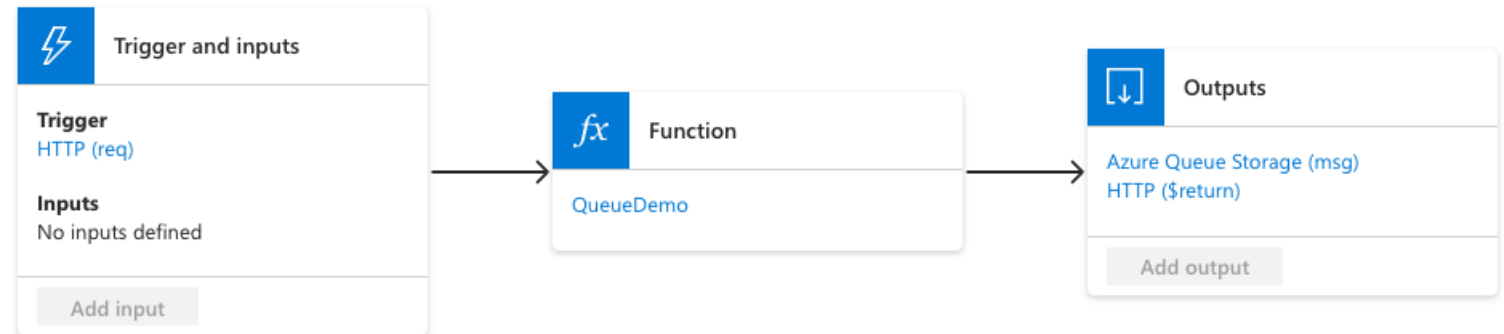
[Home](#) > [Subscriptions](#) > [p-sub-htimvs-pcls](#) | [Resource groups](#) > [pythondemofunctionpcls](#) > [python-demo-function-pcls](#) > [QueueDemo](#)

QueueDemo | Integration

python-demo-function-pcls

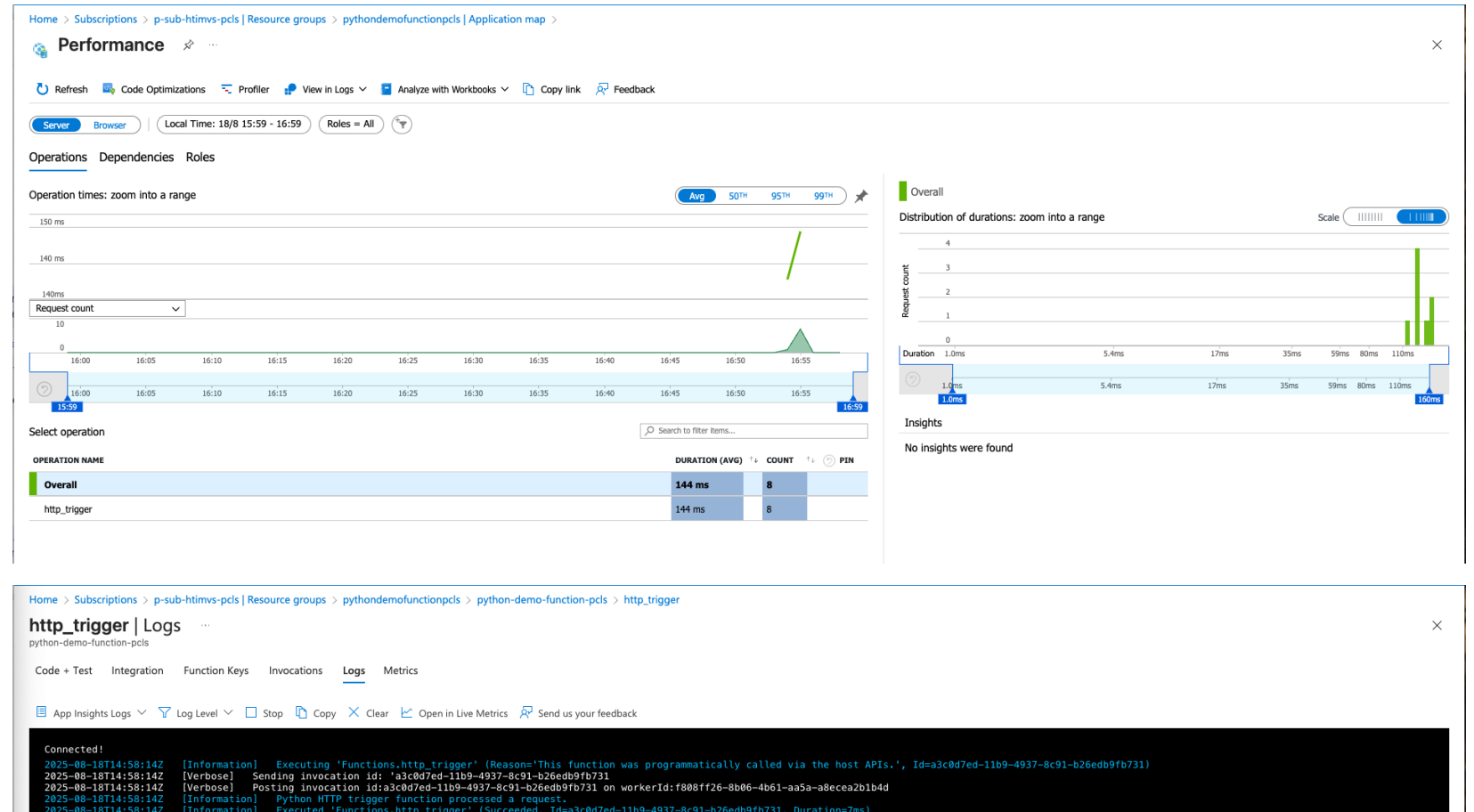
[Code + Test](#) [Integration](#) [Function Keys](#) [Invocations](#) [Logs](#) [Metrics](#)

[Refresh](#) [Send us your feedback](#)



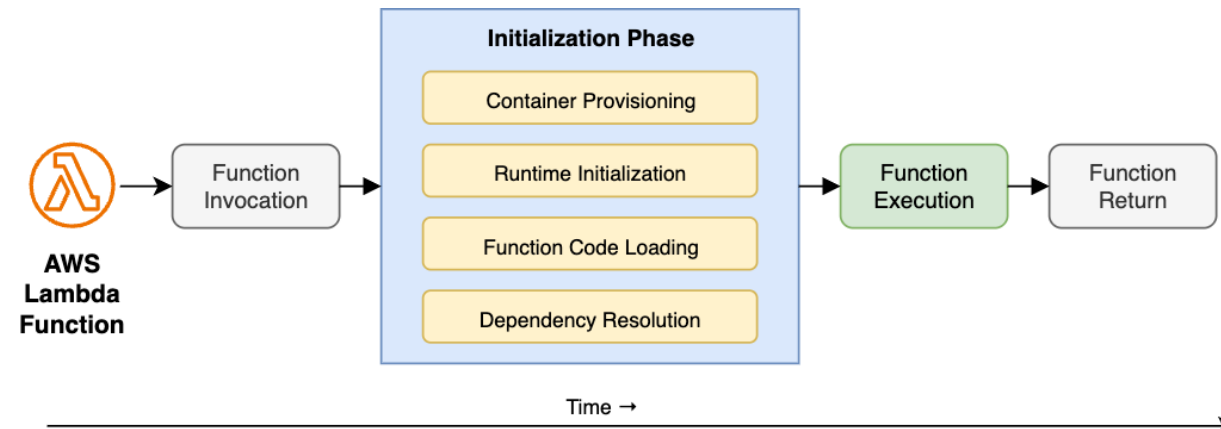
Azure Function Monitoring

- Native integration into platform monitoring solution, e.g., Azure Monitor
- Active streaming of logs possible with simultaneous testing
- Integration with OpenTelemetry possible



Cold Starts

- Cold starts are the time it takes for the function code to be loaded and executed (e.g., after no functions have been executed overnight, the runtime removes the function code from memory).
- Depends on the programming language selected (e.g., Java vs. Go).
- Mitigation measures
 - AWS Provisioned Concurrency (pre-loading of code), use Azure Premium Plan.



<https://aws.amazon.com/blogs/compute/understanding-and-remediating-cold-starts-an-aws-lambda-perspective/>

Azure Functions Pricing

- Plans with or without pre-commitment of resources
- Can be combined with other services (e.g., App Service Web Apps)
- Plan affects limits and features (i.e. cold start optimization)

<https://learn.microsoft.com/en-us/azure/azure-functions/functions/scale#service-limits>

Flex Consumption Plan

Meter	Free Grant (Per Month)	Pay as you go
On Demand Execution Time ¹	100,000 GB-s	CHF 0.000030/GB-s
On Demand Total Executions ¹	250,000 executions	CHF 0.320 per million executions
Always Ready Baseline		CHF 0.0000040/GB-s
Always Ready Execution Time		CHF 0.0000152/GB-s
Always Ready Total Executions		CHF 0.319421 per million executions

Standardization of Serverless Workflows

- Platform interoperability is virtually non-existent (e.g., migration from Azure to AWS requires refactoring)
- Standardization initiative “Serverlessworkflow” with DSL for serverless workflows (JSON/YAML)

«Serverless Workflow presents a vendor-neutral, open-source, and entirely community-driven ecosystem tailored for defining and executing DSL-based workflows in the realm of Serverless technology.»



Standardization of Serverless Workflows

```
document:
  dsl: '1.0.1'
  namespace: default
  name: sql-export-to-minio
  version: 0.1.2
do:
- exportDatabase:
  run:
    container:
      image: mcr.microsoft.com/mssql-tools
      command: >
        /bin/bash -c "sqlcmd -S $SQLSERVER_HOST -U $SQLSERVER_USER -P '$SQLSERVER_PASSWORD' -Q 'BACKUP DATABASE [!
    volumes:
      /var/backup: /backup
    environment:
      SQLSERVERHOST: sqlserver
      SQLSERVERUSER: SA
      SQLSERVERPASSWORD: P@ssw0rd
      DATABASENAME: YourDatabase
```

<https://serverlessworkflow.io/> und <https://github.com/serverlessworkflow/specification/blob/main/use-cases/automated-data-backup/README.md>

Standardization of Serverless Workflows

```
- readBackupFile:
  run:
    container:
      image: alpine
      command: >
        /bin/sh -c "cat /backup/YourDatabase.bak | base64"
      volumes:
        /var/backup: /backup
    export:
      as: '$context + { base64Backup: . }'
- uploadToMinio:
  call: http
  with:
    method: put
    endpoint:
      uri: https://minio.example.com/backups/db.bak
    authentication:
      bearer:
        token: 2548qsd5a8qsd43a
    headers:
      contentType: application/octet-stream
    body: ${ $context.base64Backup }
  schedule:
    cron: 0 0 * * *
```

<https://serverlessworkflow.io/> und <https://github.com/serverlessworkflow/specification/blob/main/use-cases/automated-data-backup/README.md>



Standardization of Events

- Specification from the CNCF Serverless WG
- Standardization of a JSON schema for events
- Wide adoption across cloud providers and open source solutions

<https://cloudevents.io/> und <https://learn.microsoft.com/en-us/azure/event-grid/cloud-event-schema>

```
{
  "specversion": "1.0",
  "type": "Microsoft.Storage.BlobCreated",
  "source": "/subscriptions/{subscription-id}/resourceGroups/{resource-group}/p",
  "id": "9aeb0fdf-c01e-0131-0922-9eb54906e209",
  "time": "2019-11-18T15:13:39.4589254Z",
  "subject": "blobServices/default/containers/{storage-container}/blobs/{new-fi",
  "data": {
    "api": "PutBlockList",
    "clientRequestId": "4c5dd7fb-2c48-4a27-bb30-5361b5de920a",
    "requestId": "9aeb0fdf-c01e-0131-0922-9eb549000000",
    "eTag": "0x8D76C39E4407333",
    "contentType": "image/png",
    "contentLength": 30699,
    "blobType": "BlockBlob",
    "url": "https://gridtesting.blob.core.windows.net/testcontainer/{new-file",
    "sequencer": "0000000000000000000000000000000099240000000000c41c18",
    "storageDiagnostics": {
      "batchId": "681fe319-3006-00a8-0022-9e7cde000000"
    }
  }
}
```

Summary

- Functions are short-lived executions of stateless program code.
- Function code requires a runtime to manage the function lifecycle and map bindings to code classes.
- Integrations in cloud platform events (e.g., bucket created or file uploaded) can trigger functions.
- Functions support scale-to-zero.



Homework

- Review the material from this lesson.
- Complete [Assignment 07](#).
- Create and deploy your own function using the following learn paths:
 - [Implement Azure Functions \(2 Module\)](#)
 - [Create serverless applications \(Module 1 bis 6\)](#)

